MODULE 28 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Fixing Slow and Expensive Queries

Finding the costly queries, reading the execution graph, bytes and guard rails, joins and skew, approximate aggregation, search indexes, and what BigQuery tunes for you.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 28 of 45

admin@sthithapragna.com

A slow query is rarely a mystery. It reads too much, moves too much, or does the same work twice.

From a big bill to a fix

01

Find it

INFORMATION_SCHEMA.JOBS, sorted by cost.

02

Read the graph

Which stage burns the slot time?

03

Cut the bytes

Columns, partition filters, guard rails.

04

Fix the shape

Joins, skew, repeated work.

05

Precompute

The ladder from module 27.

THE PATTERN

Measure before you tune. The most expensive query is often not the one people complain about.

The twenty costliest queries this week

```sql
SELECT user_email, job_id,
       total_bytes_billed / POW(1024, 4) AS tib_billed,
       total_slot_ms / 1000 / 3600      AS slot_hours,
       LEFT(query, 80)                  AS query_start
FROM `region-europe-west2`.INFORMATION_SCHEMA.JOBS
WHERE creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
  AND job_type = 'QUERY' AND state = 'DONE'
ORDER BY total_bytes_billed DESC
LIMIT 20;
```

2-3 Bytes billed drive on-demand cost. Slot hours drive capacity cost.

5 The view is regional. Name the region, or the query finds nothing.

REMEMBER

Swap JOBS for JOBS_BY_ORGANIZATION to see every project, if you hold the right role.

Reading the execution graph

- A query runs as **stages**, linked by **shuffle**. Darker nodes used more slot time.
- Each stage shows **wait, read, compute and write**, as average and maximum.
- A maximum far above the average means **skew**: a few workers did most of the work.
- Shuffle **spilled to disk** means too much data is moving between stages.

> HONEST LIMIT
>
> Dry runs and cached results have no execution graph. You need a real run to diagnose.

What the stage is telling you

YOU SEE	IT USUALLY MEANS	TRY
High **wait**	Not enough slots right now	Reservation, or another time
High **read**	Too many bytes in	Fewer columns, real partition filters
Compute **max far above avg**	Skewed key	Split or filter the hot key
Shuffle **spilled**	Huge join or grouping	Filter and aggregate first
High **write**	Huge output	Aggregate before returning

THE PATTERN

Fix the darkest stage first. Tuning a cheap stage changes nothing.

Bytes read are bytes billed

- **SELECT *** reads every column. Storage is columnar, so name only the columns you need.
- **LIMIT** does not cut the bytes read on a table that is not clustered.
- **Table preview** and bq head are free. Use them to look at data.
- Partition filters must really **prune**: no subqueries, as module 23 showed.

> FOR LARKSPUR
>
> Larkspur's analysts love SELECT star with LIMIT 10. It still reads every column of every row.

Guard rails before the bill

- A **dry run** is free, and estimates bytes. The estimate is an upper bound.
- **Maximum bytes billed** fails a query before it runs, at no charge.
- **Custom quotas** cap bytes per day, per user or per project.
- **Require partition filter** refuses queries with no usable date filter.

> THE TIEBREAKER
>
> Stop one runaway query: maximum bytes billed. Cap a user's whole day: a custom quota.

Joins: do this, avoid that

DO

Join less data

- Filter and aggregate before the join
- Largest table first, then smaller
- Small tables broadcast to every slot
- INT64 keys beat STRING keys

AVOID

Joins that explode

- Self-joins: use window functions
- Joins that output more rows than in
- ORDER BY in the middle of a query
- A WITH clause referenced many times

THE TRAP

A WITH clause may be evaluated each time it is referenced. Reused results belong in a temporary table.

A self-join, rewritten

```sql
-- Before: a self-join to find the previous half hour
SELECT a.meter_id, a.reading_ts, a.kwh - b.kwh AS change_kwh
FROM curated.readings_hh AS a JOIN curated.readings_hh AS b
  ON  b.meter_id = a.meter_id
  AND b.reading_ts = TIMESTAMP_SUB(a.reading_ts, INTERVAL 30 MINUTE)
WHERE DATE(a.reading_ts) = '2026-10-09';

-- After: one pass, with a window function
SELECT meter_id, reading_ts,
       kwh - LAG(kwh) OVER (PARTITION BY meter_id ORDER BY reading_ts) AS change_kwh
FROM curated.readings_hh
WHERE DATE(reading_ts) = '2026-10-09';
```

3 The table joined to itself: read twice, with a large shuffle.

10 LAG reads the previous row in the same pass. No join at all.

HONEST LIMIT

LAG takes the previous row, even if a reading is missing. Check for gaps if that matters.

Skew: one key, one overworked slot

- Rows with the same key go to the same worker. One huge key means **one slow worker**.
- Common culprits: **NULL**, placeholder values, or one giant customer.
- Filter placeholders out **before** the join, or handle them separately.
- Pre-aggregate the hot key, or spread it with an extra key part.

> FOR LARKSPUR
>
> Rejected readings carry meter_id 'UNKNOWN'. Joining them to meter details puts a third of the rows on one worker.

When close enough is cheaper

- **APPROX_COUNT_DISTINCT** uses HyperLogLog++, with far less memory and a small error.
- **APPROX_QUANTILES** and **APPROX_TOP_COUNT** do the same for percentiles and top lists.
- **HLL_COUNT** sketches can be stored daily, then **merged** into weeks and months.
- Bills and settlement need exact answers. Use COUNT DISTINCT there.

> THE TRAP
>
> Distinct counts cannot be added up across days. Sketches can be merged. That is their whole point.

Finding a needle in a haystack

- A **search index** speeds up point lookups in big STRING and JSON columns.
- Used by **SEARCH**, and also by =, IN and LIKE on indexed columns.
- BigQuery builds and maintains it. Index **storage** is billed.
- Not for fuzzy matching, views, external tables or time travel queries.

> THE TIEBREAKER
>
> A few rows from billions: a search index. A range of rows on a sort key: clustering.

What BigQuery tunes for you

- **History-based optimizations** learn from earlier runs of similar queries.
- Examples: semi-join reduction, join commutation, parallelism adjustment.
- Applied only with high confidence, and **revoked** if they do not help.
- See them in the jobs view, under optimization details.

> REMEMBER
>
> Automatic tuning helps, but it never fixes SELECT star or a missing partition filter.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Which queries cost the **most**?	INFORMATION_SCHEMA.JOBS, by bytes or slots
LIMIT did not lower the bill	Fewer columns, or table preview
Fail before scanning **too much**	Maximum bytes billed
Compute **max far above average**	Skew: fix the hot key
Distinct users, **day to month**	HLL_COUNT sketches, merged
One value among billions of rows	Search index and SEARCH

THE PATTERN

Too much read, too much moved, or work done twice: name which, then fix it.

> Each morning an analyst runs SELECT * with LIMIT 100 on a 30 TB unpartitioned history table, just to look at some rows.
>
> Every run is billed for the whole table.

What should the analyst do instead?

A. Add ORDER BY to the query, so that LIMIT can stop reading early

B. Use the free table preview, or bq head, to look at the rows

C. Dry run the query first each morning, then run it for real

D. Raise the project's custom quota, so the query keeps running

◆ PAUSE AND DECIDE

What should the analyst do instead?

A ORDER BY

Sorting needs every row first, so it reads even more.

B Table preview

Free, and touches no quota.

C Dry run first

Shows the cost, then pays it anyway.

D Raise the quota

Makes the waste easier, not smaller.

THE SIGNAL

Just looking at rows: preview, which is free. LIMIT does not cut bytes.

> A join of rejected readings to meter details runs for 40 minutes. In the execution graph, the join stage's maximum compute is 40 times its average.
>
> A third of the rejected readings carry the placeholder meter_id 'UNKNOWN'.

What should the team change?

A. Handle placeholder rows separately, and join only real meter IDs

B. Buy more slots, so that the join stage has more workers to use

C. Add LIMIT to the join, so that fewer rows reach the slow stage

D. Convert meter_id to a longer string, so the keys spread out better

◆ PAUSE AND DECIDE

What should the team change?

A **Handle the hot key**
Removes the skew, so the work spreads evenly.

B **More slots**
The hot key still lands on one worker.

C **LIMIT**
Changes the answer, and not the skew.

D **Longer strings**
The same key still hashes to one place.

THE SIGNAL

Max far above average, one dominant key: skew. Deal with the hot key.

> Finance wants any single analyst query that would scan more than 1 TB to fail before it runs, at no cost.
>
> Normal smaller queries must carry on as usual.

What should the team use?

A. A custom daily query quota of 1 TB for each analyst

B. Require partition filter on every table in the warehouse

C. Email analysts a reminder to dry run large queries first

D. Maximum bytes billed of 1 TB on the analysts' queries

◆ PAUSE AND DECIDE

What should the team use?

A Daily quota

Caps the whole day, not one query.

B Require partition filter

Blocks unfiltered queries, not large filtered ones.

C Reminder emails

Relies on memory, and enforces nothing.

D Maximum bytes billed

Each query over the limit fails first, at no charge.

THE SIGNAL

Stop one query before it runs, at no cost: maximum bytes billed.

> A dashboard shows distinct app users per day and per month, using COUNT(DISTINCT) over billions of events. It is slow.
>
> About 1% error is acceptable, and monthly figures should not rescan every raw event.

What should the team build?

A HLL_COUNT sketches stored per day, then merged for each month

B COUNT(DISTINCT) per day, with the daily counts added up for the month

C APPROX_COUNT_DISTINCT over the raw events for the whole month

D A Connected Sheet that counts the distinct users with a formula

◆ PAUSE AND DECIDE

What should the team build?

A Daily sketches, merged

Approximate, cheap, and months merge from days.

B Add daily counts

Counts a user once for every day they appear.

C Approximate over raw

Faster, but it rescans the whole month.

D Connected Sheets

Still runs the same heavy query underneath.

THE SIGNAL

Approximate distinct counts that roll up: HLL_COUNT sketches.

> Field support searches a 20 TB table of device logs, stored as JSON, for one device serial number, several times an hour.
>
> Each search scans the whole table, and returns a handful of rows.

What should the team add?

A Clustering on the time each log line was ingested

B Integer range partitioning on the device serial number

C A search index on the log column, queried with SEARCH

D A materialized view listing every serial number in the table

◆ PAUSE AND DECIDE

What should the team add?

A Cluster on ingest time

Helps time ranges, not one serial.

B Partition on serial

Serials are strings, and far beyond partition limits.

C Search index

Built for point lookups in big text and JSON.

D Materialized view

Still scans the logs to find the matching rows.

THE SIGNAL

A handful of rows from billions, in text or JSON: a search index.

> A long report query defines a costly WITH clause that aggregates 10 TB, then references it four times.
>
> The execution graph shows the same aggregation running four times.

What should the team change?

A Move the WITH clause to the end of the query text instead

B Write the aggregate to a temporary table once, then reuse it

C Add LIMIT inside the WITH clause, so it returns fewer rows

D Turn the WITH clause into a logical view, and query the view

◆ PAUSE AND DECIDE

What should the team change?

A **Move the clause**
Its position changes nothing about evaluation.

B **Temporary table**
Computed once, then read cheaply four times.

C **LIMIT inside**
Changes the results, and still reads ten terabytes.

D **Logical view**
A view is just saved SQL, evaluated each time too.

THE SIGNAL

The same work done several times: materialise it once.

Module 28 in five lines

- Find the cost in **INFORMATION_SCHEMA.JOBS**, then read the execution graph.
- **SELECT *** and LIMIT still read everything. Preview is free.
- **Maximum bytes billed** stops one query. Custom quotas cap a day.
- Max far above average means **skew**. Window functions beat self-joins.
- **Sketches** for distinct counts that roll up, **search indexes** for needles.

COMING UP

Next: module 29, securing analytic data in BigQuery.